```
# Acceso a Gsheets - Metodo con URL de conversion a CSV (lectura)
# El archivo debe tener permisos de lectura para TODOS

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
#################### Planilla de ventas #########################
ID_planilla_ventas= '19hLeqEN91QSI6JxUXlTvPnFeMi8EU8esSIRNhpBn3ic'

URL_ventas = f'https://docs.google.com/spreadsheets/d/{ID_planilla_ventas}/gviz/tq?tqx=out:csv&sheet='


#################### Planilla de Distribuidores #########################

ID_planilla_distribuidores= '1dYnRmsuwNt8hgvDyygWEh76ZD3gthIRObYU47knUuIo'

URL_distribuidores= f'https://docs.google.com/spreadsheets/d/{ID_planilla_distribuidores}/gviz/tq?tqx=out:csv&sheet='

#################### Planilla de Exportacion #########################

ID_planilla_expotacion= '1NAVxrAsEZgdUeZti7j4hBVGDhjjkhvwTY1-Lg1EALzM'

URL_expotacion= f'https://docs.google.com/spreadsheets/d/{ID_planilla_expotacion}/gviz/tq?tqx=out:csv&sheet='



#################### Planilla de Localizacion #########################

ID_planilla_localizacion= '1Dca7HB52JovJSMbX5ukF4JKmLXzxBszVUr40Fi3er1w'

URL_localizacion= f'https://docs.google.com/spreadsheets/d/{ID_planilla_localizacion}/gviz/tq?tqx=out:csv&sheet='


```

```
# CSV Ventas
df_Ventas= pd.read_csv(URL_ventas)
# EXPLORO LOS TIPOS DE DATOS
print (df_Ventas.dtypes)

```

```
distributor    float64
Rubber          object
Brass           object
Vinyl           object
Granite         object
Stone           object
Brick           object
Aluminum        object
Glass           object
Plexiglass      object
Steel           object
Wood            object
Plastic         object
dtype: object
```

```
#Renombro la columnas para luego poder hacer un merge a futuro
df_Ventas.rename(columns={'distributor':'Distribuidor'
,'Rubber':'Caucho'
,'Brass':'Laton'
,'Vinyl':'Vinilo'
,'Granite':'Granito'
,'Stone':'Piedra'
,'Brick':'Ladrillo'
,'Aluminum':'Aluminio'
,'Glass':'Vidrio'
,'Plexiglass':'Plexiglass'
,'Steel':'Acero'
,'Wood':'Madera'
,'Plastic':'Plastico'
}, inplace=True)
```

```
#COLUMNS NOS DESDE RUBBER HASTA LA ULTIMA COLUMNAS Y LUEGO AL LO CONVIERTO EN LISTA(con el list()) luego guardo esa lista en el campo
columnas=list(df_Ventas.columns[1:13])
print (columnas)
```

```
['Caucho', 'Laton', 'Vinilo', 'Granito', 'Piedra', 'Ladrillo', 'Aluminio', 'Vidrio', 'Plexiglass', 'Acero', 'Madera', 'Plastico']
```

```
#Elimino la Fila de Totales (Elimine la Fila 44 que contiene un string que contiene NaN)
df_Ventas.drop([44],inplace = True)
df_Ventas
```

	Distribuidor	Caucho	Laton	Vinilo	Granito	
0	583.0	$54.510.203,61	$45.268.636,86	$51.579.748,25	$21.780.180,58	$26
1	1104.0	$32.438.788,20	$25.837.100,49	$36.603.264,50	$21.883.374,92	$1
2	1384.0	$21.780.180,58	$78.927.599,01	$25.837.100,49	$33.102.840,61	$51
3	379.0	$79.358.855,35	$90.185.311,22	$45.268.636,86	$54.510.203,61	$59
4	1599.0	$11.758.005,07	$21.780.180,58	$57.187.306,41	$9.945.371,16	$32
5	1062.0	$90.185.311,22	$43.512.922,94	$11.758.005,07	$76.443.998,86	$36
6	1679.0	$78.927.599,01	$57.187.306,41	$41.899.590,44	$18.050.385,12	$20
7	1302.0	$5.662.736,92	$59.862.809,01	$46.239.695,30	$90.185.311,22	$36
8	565.0	$57.009.912,25	$53.172.624,14	$36.999.867,09	$59.199.680,31	$78
9	860.0	$36.603.264,50	$54.510.203,61	$1.346.769,34	$18.421.797,13	$10
10	707.0	$59.199.680,31	$59.199.680,31	$9.945.371,16	$55.872.547,77	$43
11	846.0	$61.236.075,66	$32.067.534,68	$21.883.374,92	$53.172.624,14	$59
12	175.0	$26.576.776,52	$22.300.838,58	$1.473.437,08	$41.572.788,80	$65
13	241.0	$59.862.809,01	$55.020.982,80	$78.927.599,01	$55.020.982,80	$54
14	1553.0	$33.102.840,61	$21.883.374,92	$43.512.922,94	$177.874.267,01	$32
15	1093.0	$1.346.769,34	$16.086.016,33	$65.978.582,24	$12.466.621,84	$11
16	1666.0	$45.268.636,86	$79.358.855,35	$59.199.680,31	$26.576.776,52	$33
17	504.0	$20.344.007,84	$1.346.769,34	$22.300.838,58	$35.563.624,11	$27
18	723.0	$77.874.267,01	$51.579.748,25	$32.067.534,68	$11.758.005,07	$57
19	1723.0	$42.411.396,23	$57.009.912,25	$90.185.311,22	$57.187.306,41	$77
20	920.0	$10.536.882,23	$27.179.878,86	$79.358.855,35	$27.009.912,25	$57
21	1055.0	$46.239.695,30	$1.473.437,08	$41.572.788,80	$61.236.075,66	$5
22	815.0	$51.579.748,25	$46.239.695,30	$53.172.624,14	$27.179.878,86	$25
23	161.0	$41.572.788,80	$35.563.624,11	$27.179.878,86	$79.358.855,35	$1
24	378.0	$16.086.016,33	$11.812.951,32	$18.050.385,12	$51.579.748,25	$21
25	1235.0	$35.563.624,11	$42.411.396,23	$81.812.951,32	$65.978.582,24	$90
26	1017.0	$22.300.838,58	$61.236.075,66	$61.236.075,66	$36.059.867,09	$61
27	1463.0	$21.883.374,92	$18.050.385,12	$59.862.809,01	$25.837.100,49	$9
28	364.0	$53.172.624,14	$33.102.840,61	$177.874.267,01	$81.812.951,32	$42
29	1710.0	$41.899.599,44	$55.872.547,77	$26.576.776,52	$41.899.590,44	$55

```
#DISTRIBUTOR AL SER UNA COLUMNA TIPO FLOAT debo transformar la columna a TIPO INT
df_Ventas['Distribuidor']=df_Ventas['Distribuidor'].astype(int)
print (df_Ventas.dtypes)
```

```
Distribuidor     int64
Caucho          object
Laton           object
Vinilo          object
Granito         object
Piedra          object
Ladrillo        object
Aluminio        object
Vidrio          object
Plexiglass      object
Acero           object
Madera          object
Plastico        object
dtype: object
```

```
#Reemplazo lo sig mediante un bucle for
#('$','') - Elimino los simbolo pesos y lo reemplazo por vacio
#('.','') - Elimino los simbolo puntos y lo reemplazo por vacio
#(',','.')- Elimino los simbolo coma y lo reemplazo por punto
#Esto me permite setear, para luego mas adelante poder realizar operaciones
for col in columnas:
 df_Ventas[col]=df_Ventas[col].replace({r'\$':'',r'\.':'',r',':'.'},regex=True)
#for col in columnas:
 #df_Ventas[col]=df_Ventas[col].str.replace(',','.')

```

```
#Ordeno de manera por Distribuidor de manera ascendente
df_Ventas.sort_values(by='Distribuidor', ascending=True)
```

	Distribuidor	Caucho	Laton	Vinilo	Granito	Piedra
43	29	65978582.24	36603264.50	57009912.25	16931446.04	41572788.80
23	161	41572788.80	35563624.11	27179878.86	79358855.35	1346769.34
12	175	26576776.52	22300838.58	1473437.08	41572788.80	65978582.24
31	234	11812951.32	11758005.07	18421797.13	43512922.94	16086016.33
13	241	59862809.01	55020982.80	78927599.01	55020982.80	54510203.61
35	308	18050385.12	41899590.44	33102840.61	20344007.84	45268636.86
41	325	36999867.09	6662942.92	76443998.86	45268636.86	18050385.12
28	364	53172624.14	33102840.61	177874267.01	81812951.32	42411396.23
24	378	16086016.33	11812951.32	18050385.12	51579748.25	21780180.58
3	379	79358855.35	90185311.22	45268636.86	54510203.61	59358855.35
17	504	20344007.84	1346769.34	22300838.58	35563624.11	27045333.20
8	565	57009912.25	53172624.14	36999867.09	59199680.31	78927599.01
0	583	54510203.61	45268636.86	51579748.25	21780180.58	26576776.52
38	619	43512922.94	20344007.84	55020982.80	42411396.23	11812951.32
10	707	59199680.31	59199680.31	9945371.16	55872547.77	43512922.94
39	715	9945371.16	41572788.80	6662942.92	1473437.08	55020982.80
18	723	77874267.01	51579748.25	32067534.68	11758005.07	57009912.25
40	808	3221765.13	10536882.23	35563624.11	46039695.30	22300838.58
22	815	51579748.25	46239695.30	53172624.14	27179878.86	25837100.49
37	818	25837100.49	26576776.52	21780180.58	6662942.92	51862809.01
11	846	61236075.66	32067534.68	21883374.92	53172624.14	59199680.31
9	860	36603264.50	54510203.61	1346769.34	18421797.13	10536882.23
20	920	10536882.23	27179878.86	79358855.35	27009912.25	57187306.41
26	1017	22300838.58	61236075.66	61236075.66	36059867.09	61236075.66
21	1055	46239695.30	1473437.08	41572788.80	61236075.66	5662736.92
5	1062	90185311.22	43512922.94	11758005.07	76443998.86	36999867.09
15	1093	1346769.34	16086016.33	65978582.24	12466621.84	11758005.07
1	1104	32438788.20	25837100.49	36603264.50	21883374.92	1473437.08
42	1169	55872547.77	18421797.13	12466621.84	59862809.01	21883374.92
34	1183	55020982.80	9945371.16	20344007.84	41927599.01	41899599.44

```
#Reemplazo el typo de datos con un astype lo sig mediante un bucle for
for col in columnas:
  df_Ventas[col]=df_Ventas[col].astype(float)
```

```
#Visualizacion de Tabla mediante el DataFrame para Detección y eliminación de duplicados y/o filas/columnas inapropiadas o innecesari
df_Ventas
pd.options.display.float_format = '{:.2f}'.format
```

```
#Visualizo que haya convertido las columnas a Float con un Dtype
df_Ventas.dtypes
```

```
Distribuidor      int64
Caucho          float64
Laton           float64
Vinilo          float64
Granito         float64
Piedra          float64
Ladrillo        float64
Aluminio        float64
Vidrio          float64
Plexiglass      float64
Acero           float64
Madera          float64
Plastico        float64
dtype: object
```

```
df_Ventas
```

	Distribuidor	Caucho	Laton	Vinilo	Granito	Piedra
0	583	54510203.61	45268636.86	51579748.25	21780180.58	26576776.52
1	1104	32438788.20	25837100.49	36603264.50	21883374.92	1473437.08
2	1384	21780180.58	78927599.01	25837100.49	33102840.61	51579748.25
3	379	79358855.35	90185311.22	45268636.86	54510203.61	59358855.35
4	1599	11758005.07	21780180.58	57187306.41	9945371.16	32067534.68
5	1062	90185311.22	43512922.94	11758005.07	76443998.86	36999867.09
6	1679	78927599.01	57187306.41	41899590.44	18050385.12	20344007.84
7	1302	5662736.92	59862809.01	46239695.30	90185311.22	36603264.50
8	565	57009912.25	53172624.14	36999867.09	59199680.31	78927599.01
9	860	36603264.50	54510203.61	1346769.34	18421797.13	10536882.23
10	707	59199680.31	59199680.31	9945371.16	55872547.77	43512922.94
11	846	61236075.66	32067534.68	21883374.92	53172624.14	59199680.31
12	175	26576776.52	22300838.58	1473437.08	41572788.80	65978582.24
13	241	59862809.01	55020982.80	78927599.01	55020982.80	54510203.61
14	1553	33102840.61	21883374.92	43512922.94	177874267.01	32438788.20
15	1093	1346769.34	16086016.33	65978582.24	12466621.84	11758005.07
16	1666	45268636.86	79358855.35	59199680.31	26576776.52	33102840.61
17	504	20344007.84	1346769.34	22300838.58	35563624.11	27045333.20
18	723	77874267.01	51579748.25	32067534.68	11758005.07	57009912.25
19	1723	42411396.23	57009912.25	90185311.22	57187306.41	77874267.01
20	920	10536882.23	27179878.86	79358855.35	27009912.25	57187306.41
21	1055	46239695.30	1473437.08	41572788.80	61236075.66	5662736.92
22	815	51579748.25	46239695.30	53172624.14	27179878.86	25837100.49
23	161	41572788.80	35563624.11	27179878.86	79358855.35	1346769.34
24	378	16086016.33	11812951.32	18050385.12	51579748.25	21780180.58
25	1235	35563624.11	42411396.23	81812951.32	65978582.24	90185311.22
26	1017	22300838.58	61236075.66	61236075.66	36059867.09	61236075.66
27	1463	21883374.92	18050385.12	59862809.01	25837100.49	9945371.16
28	364	53172624.14	33102840.61	177874267.01	81812951.32	42411396.23
29	1710	41899599.44	55872547.77	26576776.52	41899590.44	55072547.77

```
#Saco las metricas del df_Ventas
df_Ventas[columnas].describe()
```

	Caucho	Laton	Vinilo	Granito	Piedra	Ladrillo	
count	44.00	44.00	44.00	44.00	44.00	44.00	
mean	38192135.03	41833167.75	43467934.56	41938512.51	37534531.72	3469.50	4
std	22861617.98	30637106.50	30812148.98	30316025.99	22046995.00	9185.90	3
min	1346769.34	1346769.34	1346769.34	1346769.34	1346769.34	0.00	
25%	21421137.39	21421137.39	21857576.34	21857576.34	21421137.39	0.00	2
50%	36801565.80	39286327.95	41736189.62	39088026.65	36801565.80	0.00	4
75%	55233874.04	56156888.89	57054260.79	55233874.04	54637898.41	0.00	5
max	90185311.22	177874267.01	177874267.01	177874267.01	90185311.22	42458.00	17

```
#Saco la sumatoria de todas las columnas
df_Ventas[columnas].sum()
```

```
Caucho        1680453941.29
Laton         1840659380.95
Vinilo        1912589120.56
Granito       1845294550.27
Piedra        1651519395.63
Ladrillo          152658.00
Aluminio      2163832570.85
Vidrio        2173275882.75
Plexiglass   25974040668.70
Acero               58457.97
Madera        1930565060.44
Plastico      2047338174.32
dtype: float64
```

```
#Saco la sumatoria de todas las columnas y lo almaceno en una variable para luego poder graficar
total_sumasventas=df_Ventas[columnas].sum()
print(total_sumasventas)
```

```
Caucho        1680453941.29
Laton         1840659380.95
Vinilo        1912589120.56
Granito       1845294550.27
Piedra        1651519395.63
Ladrillo          152658.00
Aluminio      2163832570.85
Vidrio        2173275882.75
Plexiglass   25974040668.70
Acero               58457.97
Madera        1930565060.44
Plastico      2047338174.32
dtype: float64
```

```
df_Ventas
```

	Distribuidor	Caucho	Laton	Vinilo	Granito	Piedra
0	583	54510203.61	45268636.86	51579748.25	21780180.58	26576776.52
1	1104	32438788.20	25837100.49	36603264.50	21883374.92	1473437.08
2	1384	21780180.58	78927599.01	25837100.49	33102840.61	51579748.25
3	379	79358855.35	90185311.22	45268636.86	54510203.61	59358855.35
4	1599	11758005.07	21780180.58	57187306.41	9945371.16	32067534.68
5	1062	90185311.22	43512922.94	11758005.07	76443998.86	36999867.09
6	1679	78927599.01	57187306.41	41899590.44	18050385.12	20344007.84
7	1302	5662736.92	59862809.01	46239695.30	90185311.22	36603264.50
8	565	57009912.25	53172624.14	36999867.09	59199680.31	78927599.01
9	860	36603264.50	54510203.61	1346769.34	18421797.13	10536882.23
10	707	59199680.31	59199680.31	9945371.16	55872547.77	43512922.94
11	846	61236075.66	32067534.68	21883374.92	53172624.14	59199680.31
12	175	26576776.52	22300838.58	1473437.08	41572788.80	65978582.24
13	241	59862809.01	55020982.80	78927599.01	55020982.80	54510203.61
14	1553	33102840.61	21883374.92	43512922.94	177874267.01	32438788.20
15	1093	1346769.34	16086016.33	65978582.24	12466621.84	11758005.07
16	1666	45268636.86	79358855.35	59199680.31	26576776.52	33102840.61
17	504	20344007.84	1346769.34	22300838.58	35563624.11	27045333.20
18	723	77874267.01	51579748.25	32067534.68	11758005.07	57009912.25
19	1723	42411396.23	57009912.25	90185311.22	57187306.41	77874267.01
20	920	10536882.23	27179878.86	79358855.35	27009912.25	57187306.41
21	1055	46239695.30	1473437.08	41572788.80	61236075.66	5662736.92
22	815	51579748.25	46239695.30	53172624.14	27179878.86	25837100.49
23	161	41572788.80	35563624.11	27179878.86	79358855.35	1346769.34
24	378	16086016.33	11812951.32	18050385.12	51579748.25	21780180.58
25	1235	35563624.11	42411396.23	81812951.32	65978582.24	90185311.22
26	1017	22300838.58	61236075.66	61236075.66	36059867.09	61236075.66
27	1463	21883374.92	18050385.12	59862809.01	25837100.49	9945371.16
28	364	53172624.14	33102840.61	177874267.01	81812951.32	42411396.23
29	1710	41899599.44	55872547.77	26576776.52	41899590.44	55072547.77

```
##################################################################################################################


################################################################ EXPORTACION ################################################################


##################################################################################################################
```

```
# CSV Exportacion
df_Exportacion= pd.read_csv(URL_expotacion, thousands = '.')

# EXPLORO LOS TIPOS DE DATOS
print (df_Exportacion.dtypes)
```

```
distributor      int64
Rubber          object
Brass           object
Vinyl           object
Granite         object
Stone           object
Brick           object
Aluminum        object
Glass           object
Plexiglass      object
Steel           object
Wood            object
Plastic         object
Unnamed: 13    float64
Unnamed: 14     object
Columnas        object
dtype: object
```

```
#Renombro la columnas para luego poder hacer un merge a futuro
df_Exportacion.rename(columns={'distributor':'Distribuidor'
,'Rubber':'Caucho'
,'Brass':'Laton'
,'Vinyl':'Vinilo'
,'Granite':'Granito'
,'Stone':'Piedra'
,'Brick':'Ladrillo'
,'Aluminum':'Aluminio'
,'Glass':'Vidrio'
,'Plexiglass':'Plexiglass'
,'Steel':'Acero'
,'Wood':'Madera'
,'Plastic':'Plastico'
}, inplace=True)
```

```
#Imprimo para luego ver los datos
df_Exportacion
```

	Distribuidor	Caucho	Laton	Vinilo	Granito	
0	1526	$22.431.099,00	$36.031.577,00	$31.118.167,00	$21.322.223,00	$35.38
1	1553	$27.566.922,00	$21.996.538,00	$39.412.316,00	$25.681.987,00	$41.86
2	1666	$37.577.095,00	$41.457.655,00	$31.467.967,00	$37.577.926,00	$35.84
3	364	$36.012.730,00	$41.667.692,00	$22.837.073,00	$29.288.200,00	$39.55
4	920	$43.416.417,00	$36.290.780,00	$23.679.738,00	$21.183.706,00	$25.21
5	860	$43.909.858,00	$27.128.183,00	$36.391.549,00	$21.542.467,00	$34.97
6	241	$28.550.182,00	$36.499.732,00	$41.512.708,00	$22.262.065,00	$30.27
7	175	$21.771.886,00	$43.109.659,00	$23.247.953,00	$22.866.072,00	$31.45
8	308	$36.667.511,00	$25.366.105,00	$24.902.331,00	$36.605.735,00	$44.67
9	1183	$39.958.024,00	$40.169.347,00	$35.399.555,00	$25.461.219,00	$37.80
10	1710	$23.722.590,00	$24.623.967,00	$41.133.545,00	$35.578.933,00	$45.01
11	325	$25.138.331,00	$38.634.366,00	$35.724.628,00	$21.355.595,00	$42.95
12	234	$37.817.347,00	$23.272.550,00	$36.228.786,00	$40.908.298,00	$44.28
13	1169	$38.180.046,00	$28.960.197,00	$38.400.970,00	$23.646.886,00	$39.38
14	1062	$21.870.115,00	$29.728.962,00	$29.018.088,00	$21.585.132,00	$44.76
15	583	$28.415.088,00	$38.304.440,00	$32.154.709,00	$23.047.802,00	$23.12
16	565	$23.507.965,00	$41.845.467,00	$42.055.834,00	$35.453.957,00	$27.27
17	846	$33.377.107,00	$29.404.830,00	$37.302.875,00	$38.573.860,00	$27.52
18	504	$40.875.271,00	$22.374.529,00	$20.935.764,00	$31.145.028,00	$25.31
19	1169	$38.180.046,00	$28.960.197,00	$38.400.970,00	$23.646.886,00	$39.38
20	1463	$30.129.379,00	$36.728.723,00	$32.550.897,00	$20.871.609,00	$44.75
21	707	$31.142.709,00	$28.352.801,00	$28.274.723,00	$43.010.066,00	$44.66
22	818	$22.957.310,00	$24.387.893,00	$41.579.620,00	$26.804.859,00	$37.69
23	1599	$28.135.181,00	$23.695.417,00	$32.743.214,00	$23.511.170,00	$29.70
24	1384	$34.544.298,00	$27.882.865,00	$22.664.200,00	$21.057.646,00	$22.59
25	1723	$28.318.181,00	$23.132.460,00	$29.998.057,00	$42.240.878,00	$25.72
26	1235	$37.794.819,00	$26.812.043,00	$25.270.289,00	$28.139.664,00	$26.17
27	378	$35.622.161,00	$22.845.538,00	$31.984.270,00	$32.821.654,00	$31.74
28	1418	$42.018.216,00	$21.936.967,00	$36.106.133,00	$29.906.692,00	$25.72
29	1017	$34.349.322,00	$20.737.841,00	$26.603.366,00	$32.141.549,00	$31.81

```
#Observamos la Columna 13, 14, y Columnas que no contienen datos y debemos eliminar
#Elimino con el DROPNA todas las columnas que contengan un dato NA (Elimine la columna unnamed y Columnas )
df_Exportacion.dropna(axis=1, inplace = True)
df_Exportacion
```

	Distribuidor	Caucho	Laton	Vinilo	Granito	
0	1526	$22.431.099,00	$36.031.577,00	$31.118.167,00	$21.322.223,00	$35.38
1	1553	$27.566.922,00	$21.996.538,00	$39.412.316,00	$25.681.987,00	$41.86
2	1666	$37.577.095,00	$41.457.655,00	$31.467.967,00	$37.577.926,00	$35.84
3	364	$36.012.730,00	$41.667.692,00	$22.837.073,00	$29.288.200,00	$39.55
4	920	$43.416.417,00	$36.290.780,00	$23.679.738,00	$21.183.706,00	$25.21
5	860	$43.909.858,00	$27.128.183,00	$36.391.549,00	$21.542.467,00	$34.97
6	241	$28.550.182,00	$36.499.732,00	$41.512.708,00	$22.262.065,00	$30.27
7	175	$21.771.886,00	$43.109.659,00	$23.247.953,00	$22.866.072,00	$31.45
8	308	$36.667.511,00	$25.366.105,00	$24.902.331,00	$36.605.735,00	$44.67
9	1183	$39.958.024,00	$40.169.347,00	$35.399.555,00	$25.461.219,00	$37.80
10	1710	$23.722.590,00	$24.623.967,00	$41.133.545,00	$35.578.933,00	$45.01
11	325	$25.138.331,00	$38.634.366,00	$35.724.628,00	$21.355.595,00	$42.95
12	234	$37.817.347,00	$23.272.550,00	$36.228.786,00	$40.908.298,00	$44.28
13	1169	$38.180.046,00	$28.960.197,00	$38.400.970,00	$23.646.886,00	$39.38
14	1062	$21.870.115,00	$29.728.962,00	$29.018.088,00	$21.585.132,00	$44.76
15	583	$28.415.088,00	$38.304.440,00	$32.154.709,00	$23.047.802,00	$23.12
16	565	$23.507.965,00	$41.845.467,00	$42.055.834,00	$35.453.957,00	$27.27
17	846	$33.377.107,00	$29.404.830,00	$37.302.875,00	$38.573.860,00	$27.52
18	504	$40.875.271,00	$22.374.529,00	$20.935.764,00	$31.145.028,00	$25.31
19	1169	$38.180.046,00	$28.960.197,00	$38.400.970,00	$23.646.886,00	$39.38
20	1463	$30.129.379,00	$36.728.723,00	$32.550.897,00	$20.871.609,00	$44.75
21	707	$31.142.709,00	$28.352.801,00	$28.274.723,00	$43.010.066,00	$44.66
22	818	$22.957.310,00	$24.387.893,00	$41.579.620,00	$26.804.859,00	$37.69
23	1599	$28.135.181,00	$23.695.417,00	$32.743.214,00	$23.511.170,00	$29.70
24	1384	$34.544.298,00	$27.882.865,00	$22.664.200,00	$21.057.646,00	$22.59
25	1723	$28.318.181,00	$23.132.460,00	$29.998.057,00	$42.240.878,00	$25.72
26	1235	$37.794.819,00	$26.812.043,00	$25.270.289,00	$28.139.664,00	$26.17
27	378	$35.622.161,00	$22.845.538,00	$31.984.270,00	$32.821.654,00	$31.74
28	1418	$42.018.216,00	$21.936.967,00	$36.106.133,00	$29.906.692,00	$25.72
29	1017	$34.349.322,00	$20.737.841,00	$26.603.366,00	$32.141.549,00	$31.81

```
#Observo que los numeros de Distribuidor contienen punto, y de Rubber hasta la ultima columna Plastic tienen simbolos como "$", "." ,
#Reemplazo lo sig mediante un bucle for
#('$','') - Elimino los simbolo pesos y lo reemplazo por vacio
#('.','') - Elimino los simbolo puntos y lo reemplazo por vacio
#(',','.')- Elimino los simbolo coma y lo reemplazo por punto
#Esto me permite setear, para luego mas adelante poder realizar operaciones
for col in columnas:
 df_Exportacion[col]=df_Exportacion[col].str.replace('$','')
 df_Exportacion[col]=df_Exportacion[col].str.replace('.','')
for col in columnas:
 df_Exportacion[col]=df_Exportacion[col].str.replace(',','.')
```

```
#Verifico que no hayan datos duplicados y que me los sume (Me indica que la fila 19 se encuentra duplicada y la debo eliminar)
df_Exportacion.duplicated()
```

```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9     False
10    False
11    False
12    False
13    False
14    False
15    False
16    False
17    False
18    False
19     True
20    False
21    False
22    False
23    False
24    False
25    False
26    False
27    False
28    False
29    False
30    False
31    False
32    False
33    False
34    False
35    False
36    False
37    False
38    False
39    False
40    False
41     True
42    False
43    False
44    False
45    False
dtype: bool
```

```
#Muestro los datos duplicados por Distribuidor
df_Exportacion[df_Exportacion['Distribuidor'].duplicated(keep=False)]
```

	Distribuidor	Caucho	Laton	Vinilo	Granito	Piedra	
11	325	25138331.00	38634366.00	35724628.00	21355595.00	42958842.00	270
13	1169	38180046.00	28960197.00	38400970.00	23646886.00	39383770.00	1227
19	1169	38180046.00	28960197.00	38400970.00	23646886.00	39383770.00	1227
41	325	25138331.00	38634366.00	35724628.00	21355595.00	42958842.00	270

```
#Borro la fila 19 y 41 se encuentra duplicada
df_Exportacion=df_Exportacion.drop_duplicates()
```

```
#TENGO QUE CAMBIAR LAS COLUMNAS DE OBJETO A FLOTANTE PARA PODER REALIZAR OPERACIONES
#VISUALIZO EL TIPO DE FORMATO EN LAS COLUMNAS O VARIABLES QUE VAN DESDE RUBBER HASTA PLASTIC EN OBJECT
df_Exportacion.dtypes
```

```
Distribuidor     int64
Caucho          object
```

```
Laton           object
Vinilo          object
Granito         object
Piedra          object
Ladrillo        object
Aluminio        object
Vidrio          object
Plexiglass      object
Acero           object
Madera          object
Plastico        object
dtype: object
```

```
#HAGO LA CONVERSION DE LAS COLUMNAS A FLOAT 0 A 9 Y DISTRIBUTOR LO TENGO QUE DEJAR COMO INT PARA PODER UNIRLO AL CSV VENTAS
for col in columnas:
  df_Exportacion[col]=df_Exportacion[col].astype(float)

df_Exportacion['Distribuidor']=df_Exportacion['Distribuidor'].astype(int)
```

```
<ipython-input-34-69c945587cb8>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers
  df_Exportacion[col]=df_Exportacion[col].astype(float)
<ipython-input-34-69c945587cb8>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers
  df_Exportacion[col]=df_Exportacion[col].astype(float)
<ipython-input-34-69c945587cb8>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers
  df_Exportacion[col]=df_Exportacion[col].astype(float)
<ipython-input-34-69c945587cb8>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers
  df_Exportacion[col]=df_Exportacion[col].astype(float)
<ipython-input-34-69c945587cb8>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers
  df_Exportacion[col]=df_Exportacion[col].astype(float)
<ipython-input-34-69c945587cb8>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers
  df_Exportacion[col]=df_Exportacion[col].astype(float)
<ipython-input-34-69c945587cb8>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers
  df_Exportacion[col]=df_Exportacion[col].astype(float)
<ipython-input-34-69c945587cb8>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers
  df_Exportacion[col]=df_Exportacion[col].astype(float)
<ipython-input-34-69c945587cb8>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
#Ordeno el df.Exportacion por distribuidor de manera ascendente y fijo los cambios
df_Exportacion.sort_values(by='Distribuidor', ascending= True, inplace=True)
```

```
<ipython-input-35-5704638af537>:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus
  df_Exportacion.sort_values(by='Distribuidor', ascending= True, inplace=True)
```

```
df_Exportacion
```

	Distribuidor	Caucho	Laton	Vinilo	Granito	Piedra
38	29	43576523.00	30181558.00	32773382.00	36669396.00	45128426.00
35	161	20607246.00	23005062.00	27506653.00	26062955.00	27696836.00
7	175	21771886.00	43109659.00	23247953.00	22866072.00	31459461.00
12	234	37817347.00	23272550.00	36228786.00	40908298.00	44287587.00
6	241	28550182.00	36499732.00	41512708.00	22262065.00	30270432.00
8	308	36667511.00	25366105.00	24902331.00	36605735.00	44673772.00
11	325	25138331.00	38634366.00	35724628.00	21355595.00	42958842.00
3	364	36012730.00	41667692.00	22837073.00	29288200.00	39553494.00
27	378	35622161.00	22845538.00	31984270.00	32821654.00	31742905.00
39	379	42160913.00	41409478.00	37110518.00	25210663.00	28016394.00
18	504	40875271.00	22374529.00	20935764.00	31145028.00	25311142.00
16	565	23507965.00	41845467.00	42055834.00	35453957.00	27278926.00
15	583	28415088.00	38304440.00	32154709.00	23047802.00	23121310.00
42	619	41761417.00	33709306.00	27497587.00	28311310.00	21683294.00
21	707	31142709.00	28352801.00	28274723.00	43010066.00	44667972.00
43	715	20880116.00	37111825.00	34422161.00	40759561.00	26050968.00
37	723	29585885.00	42849169.00	30521280.00	21745658.00	32580700.00
30	808	44229593.00	42500506.00	33478392.00	37522478.00	32650463.00
36	815	32853600.00	43936493.00	31944768.00	23457213.00	30668319.00
22	818	22957310.00	24387893.00	41579620.00	26804859.00	37698018.00
17	846	33377107.00	29404830.00	37302875.00	38573860.00	27524149.00
5	860	43909858.00	27128183.00	36391549.00	21542467.00	34971118.00
4	920	43416417.00	36290780.00	23679738.00	21183706.00	25210622.00
29	1017	34349322.00	20737841.00	26603366.00	32141549.00	31818580.00
33	1055	35858240.00	22072249.00	21646802.00	39650737.00	40985840.00
14	1062	21870115.00	29728962.00	29018088.00	21585132.00	44760537.00
45	1093	35580430.00	39978595.00	32046855.00	30147636.00	36212747.00
32	1104	41387139.00	32973505.00	38520126.00	37505722.00	32781557.00
13	1169	38180046.00	28960197.00	38400970.00	23646886.00	39383770.00
9	1183	39958024.00	40169347.00	35399555.00	25461219.00	37801759.00

```
#Saco las metricas de Exportacion
df_Exportacion.describe()
```

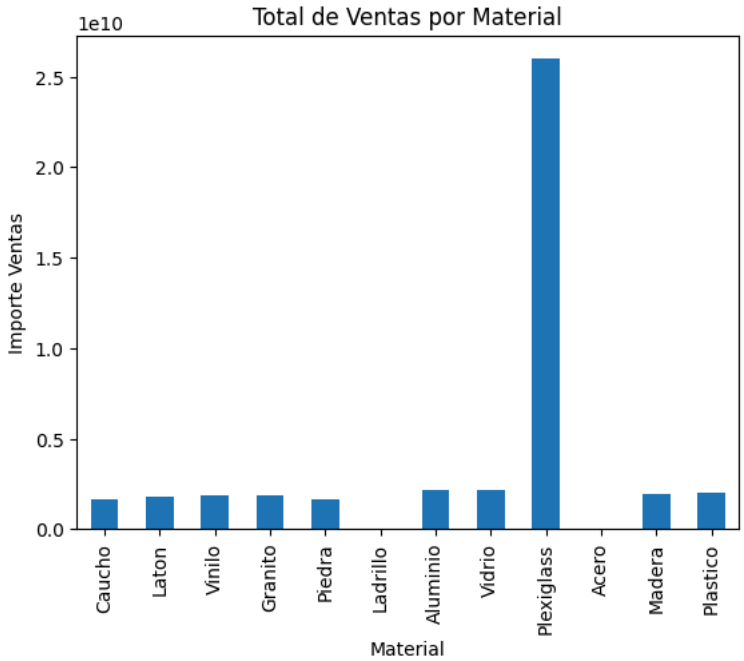
	Distribuidor	Caucho	Laton	Vinilo	Granito	Piedra
count	44.00	44.00	44.00	44.00	44.00	44.00
mean	933.34	33296420.05	31691793.75	31728419.39	30390226.45	33512974.75
std	500.12	7292618.57	7868101.75	5806897.43	7851079.25	7317395.01
min	29.00	20607246.00	20737841.00	20935764.00	20871609.00	21683294.00
25%	549.75	28051209.50	23725491.25	27362197.00	23233564.25	27187189.25
50%	890.00	34446810.00	29940461.50	32015562.50	28799755.00	32615581.50
75%	1392.50	38624540.50	38970423.25	36136796.25	37509911.00	39426201.00
max	1723.00	44229593.00	44568676.00	42055834.00	45198396.00	45128426.00

```
#Saco los totales de Exportacion
df_Exportacion[columnas].sum()
```

```
Caucho        1465042482.00
Laton         1394438925.00
Vinilo        1396050453.00
Granito       1337169964.00
Piedra        1474570889.00
Ladrillo      5260549009.00
Aluminio      1513169630.00
Vidrio        1429786765.00
Plexiglass   20779232535.00
Acero        13652930026.00
Madera        1409171577.00
Plastico      1456143794.00
dtype: float64
```

```
#Grafico de Suma de Ventas
plt.title ("Total de Ventas por Material")
plt.xlabel("Material")
plt.ylabel("Importe Ventas")
series = pd.Series(total_sumasventas)
series.plot.bar()
```

```
<Axes: title={'center': 'Total de Ventas por Material'}, xlabel='Material',
ylabel='Importe Ventas'>
```



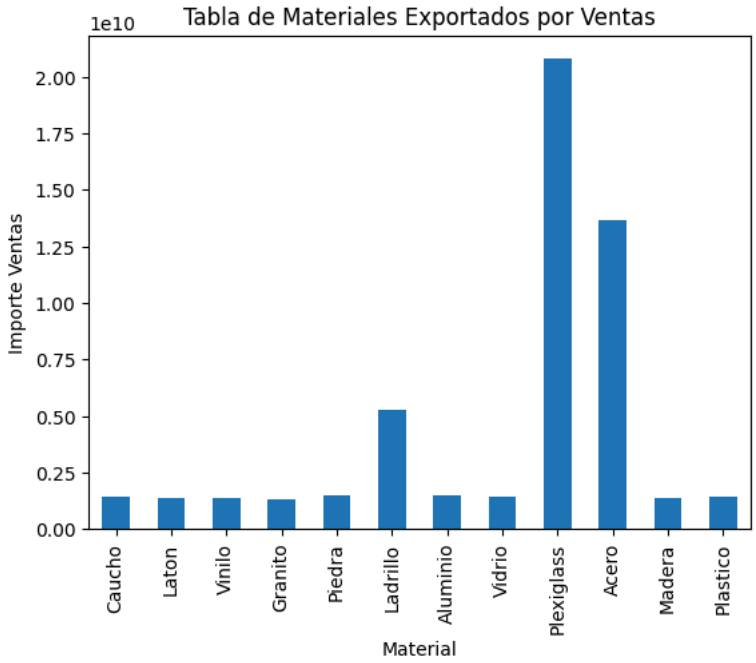

```
#Totales de Exportacion lo almaceno en una variable
total_sumasexportacion=df_Exportacion[columnas].sum()
print (total_sumasexportacion)
```

```
Caucho        1465042482.00
Laton         1394438925.00
Vinilo        1396050453.00
Granito       1337169964.00
Piedra        1474570889.00
Ladrillo      5260549009.00
Aluminio      1513169630.00
Vidrio        1429786765.00
Plexiglass   20779232535.00
```

```
Acero        13652930026.00
Madera        1409171577.00
Plastico      1456143794.00
dtype: float64
```

```
plt.xlabel("Material")
plt.ylabel("Importe Ventas")
plt.title("Tabla de Materiales Exportados por Ventas")
series = pd.Series(total_sumasexportacion)
series.plot.bar()
```

```
<Axes: title={'center': 'Tabla de Materiales Exportados por Ventas'},
xlabel='Material', ylabel='Importe Ventas'>
```



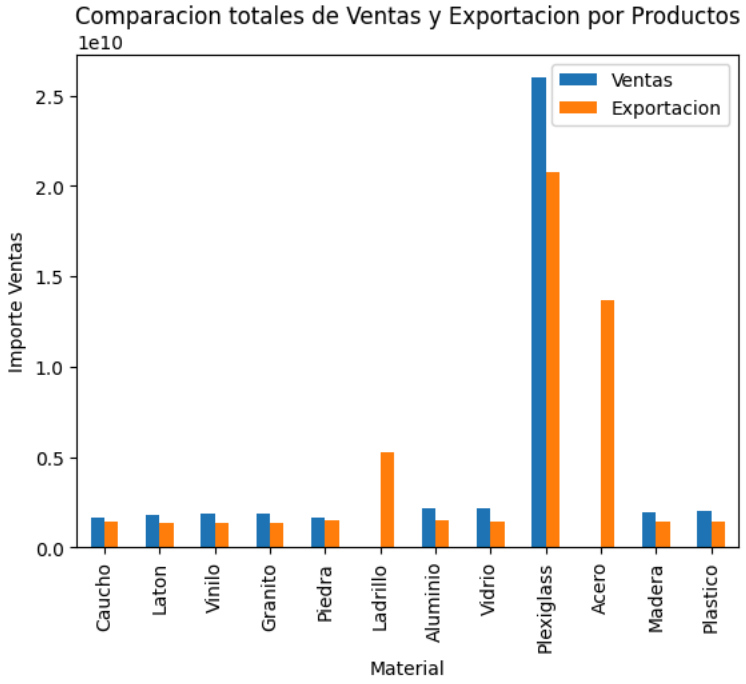

```
#Concateno la serie de total_sumasventas y total_sumasexportacion en una DataFrame union_frame
union_frame = pd.concat([total_sumasventas,total_sumasexportacion ], axis=1)
```

```
union_frame.rename(columns={0:'Ventas',1:'Exportacion'}, inplace=True)
union_frame
```

	Ventas	Exportacion
Caucho	1680453941.29	1465042482.00
Laton	1840659380.95	1394438925.00
Vinilo	1912589120.56	1396050453.00
Granito	1845294550.27	1337169964.00
Piedra	1651519395.63	1474570889.00
Ladrillo	152658.00	5260549009.00
Aluminio	2163832570.85	1513169630.00
Vidrio	2173275882.75	1429786765.00
Plexiglass	25974040668.70	20779232535.00
Acero	58457.97	13652930026.00
Madera	1930565060.44	1409171577.00
Plastico	2047338174.32	1456143794.00

```
#Relizo mediante los graficos de data frame plot.bar() un grafico de barra que me muestre
union_frame.plot.bar()
plt.xlabel("Material")
plt.ylabel("Importe Ventas")
plt.title("Comparacion totales de Ventas y Exportacion por Productos")
```

```
Text(0.5, 1.0, 'Comparacion totales de Ventas y Exportacion por Productos')
```




```
#Sumo las columas de Todos los Productos
union_frame.sum()
```

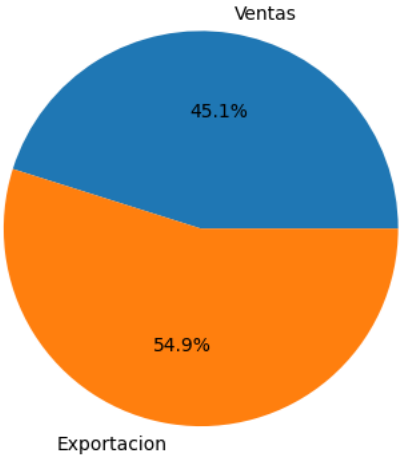
```
Ventas        43219779861.73
Exportacion   52568256049.00
dtype: float64
```

```
#Almaceno el total de Ventas de productos y total de exportaciones
ventas_exportacion=union_frame.sum()
print (ventas_exportacion)
```

```
Ventas        43219779861.73
Exportacion   52568256049.00
dtype: float64
```

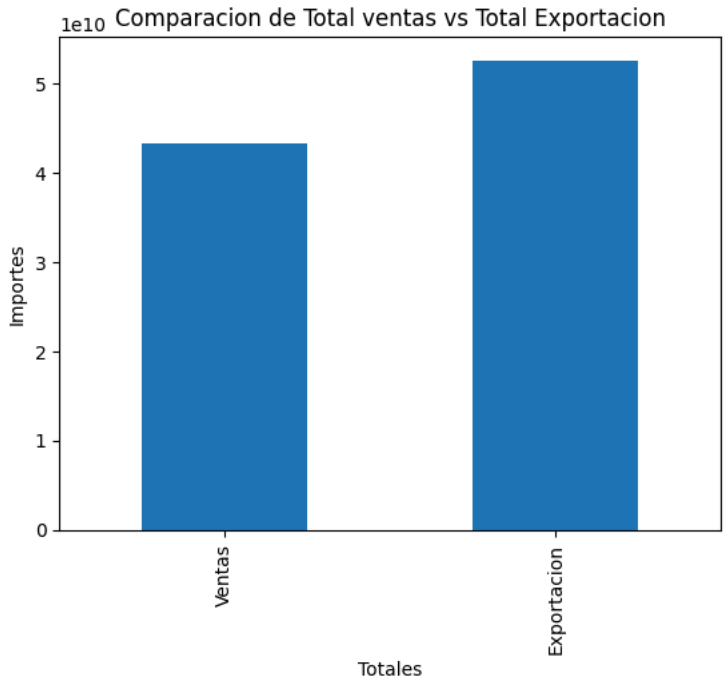
```
#Muestro el total de Ventas vs Exportacion mediante un grafico de torta el argumento autopct colocar los valores porcentuales dentro
#% saca el porcentaje de las variables y luego es el signo 1.1f lo convierte a decimal
ventas_exportacion['Ventas':'Exportacion'].plot.pie(autopct='%1.1f%%')

```

```
<Axes: >
```




```
ventas_exportacion.plot.bar()
plt.xlabel("Totales")
plt.ylabel("Importes")
plt.title("Comparacion de Total ventas vs Total Exportacion")
```

```
Text(0.5, 1.0, 'Comparacion de Total ventas vs Total Exportacion')
```




```
################################################################################################################
################################################################ DISTRIBUIDORES ################################################################
################################################################################################################
```

```
# CSV Distribuidores
df_Distribuidores= pd.read_csv(URL_distribuidores)
# EXPLORO LOS TIPOS DE DATOS
print (df_Distribuidores.dtypes)
```

```
id                                  float64
distributor                          object
distributor activities               object
years in the construction market      int64
dtype: object
```

```
#IMPRIMO PARA VER QUE DATOS TENGO QUE BORRAR y SETEAR
df_Distribuidores
```

	id	distributor	distributor activities	years in the construction market
0	29.00	Cummings-Ward	construction materials import/distribution, ir...	23
1	161.00	Maggio-Prohaska	construction materials import/distribution, ne...	20
2	175.00	Legros Group	construction materials import/distribution, st...	23
3	234.00	Koepp, Hessel and Runolfsson	construction materials import/distribution	15
4	241.00	Koch, Lang and Mertz	construction materials import/distribution, po...	12
5	308.00	Reichel and Sons	construction materials import/distribution	16
6	325.00	Marks Inc	construction materials import/distribution	19
7	364.00	Okuneva, Simonis and Hintz	construction materials import/distribution	24
8	378.00	Schiller-Bartoletti	construction materials import/distribution	17
9	379.00	Gerlach Group	construction materials import/distribution, el...	2
10	504.00	Herman, Turcotte and Osinski	construction materials import/distribution, ce...	16
11	565.00	Abernathy-Hayes	construction materials import/distribution, ir...	15
12	583.00	Kulas-Terry	construction materials import/distribution, co...	19
13	619.00	Kerluke, Barrows and Murazik	construction materials import/distribution, as...	19
14	707.00	Haley-Larson	construction materials import/distribution	14
15	715.00	Keebler, Deckow and Watsica	construction materials import/distribution	8
16	723.00	Stehr-Mitchell	construction materials import/distribution	9
17	808.00	Reilly, Hickle and Thiel	construction materials import/distribution, ne...	18
18	815.00	Schamberger and Sons	construction materials import/distribution, co...	8
		Grady, Kessler and	construction materials	

```
#Verifico que no hayan datos duplicados y que me los sume (Me indica que la fila 19 se encuentra duplicada y la debo eliminar)
df_Distribuidores.duplicated()
```

```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9     False
10    False
11    False
12    False
13    False
14    False
15    False
16    False
17    False
18    False
19    False
20    False
21    False
22    False
23    False
24    False
25    False
26    False
27    False
28    False
29    False
30    False
31    False
32    False
33    False
34    False
35    False
36    False
37    False
38    False
39    False
40    False
41    False
42    False
43    False
44    False
45     True
dtype: bool
```

```
#Observamos la Columna 1, FILA 44 Y 45 son columnas que no tienen un dato NaN que esta duplicado
#Muestro los datos duplicados por ID
df_Distribuidores[df_Distribuidores['id'].duplicated(keep=False)]
```

	id	distributor	distributor activities	years in the construction market
44	NaN	NaN	NaN	0
45	NaN	NaN	NaN	0

```
#Elimino la fila 44 y 45 que no los datos estan duplicados y no contienen Informacion
df_Distribuidores.drop([44,45],inplace = True)
```

```
#Para dejar correctamente la columna ID hago un casteo los valores que son menor a 2 por ejemplo 1.500, los multiplico *1000 para lue
df_Distribuidores.loc[df_Distribuidores['id']<2,'id']*=1000
df_Distribuidores
```

	id	distributor	distributor activities	years in the construction market
0	29.00	Cummings-Ward	construction materials import/distribution, ir...	23
1	161.00	Maggio-Prohaska	construction materials import/distribution, ne...	20
2	175.00	Legros Group	construction materials import/distribution, st...	23
3	234.00	Koepp, Hessel and Runolfsson	construction materials import/distribution	15
4	241.00	Koch, Lang and Mertz	construction materials import/distribution, po...	12
5	308.00	Reichel and Sons	construction materials import/distribution	16
6	325.00	Marks Inc	construction materials import/distribution	19
7	364.00	Okuneva, Simonis and Hintz	construction materials import/distribution	24
8	378.00	Schiller-Bartoletti	construction materials import/distribution	17
9	379.00	Gerlach Group	construction materials import/distribution, el...	2
10	504.00	Herman, Turcotte and Osinski	construction materials import/distribution, ce...	16
11	565.00	Abernathy-Hayes	construction materials import/distribution, ir...	15
12	583.00	Kulas-Terry	construction materials import/distribution, co...	19
13	619.00	Kerluke, Barrows and Murazik	construction materials import/distribution, as...	19
14	707.00	Haley-Larson	construction materials import/distribution	14
15	715.00	Keebler, Deckow and Watsica	construction materials import/distribution	8
16	723.00	Stehr-Mitchell	construction materials import/distribution	9
17	808.00	Reilly, Hickle and Thiel	construction materials import/distribution, ne...	18
18	815.00	Schamberger and Sons	construction materials import/distribution, co...	8
		Grady, Kessler and	construction materials	

```
#Cambio el TIPO DE COLUMNA ID QUE ESTA EN FLOAT A INT de la columna ID a INT
df_Distribuidores['id']=df_Distribuidores['id'].astype(int)
df_Distribuidores
```

	id	distributor	distributor activities	years in the construction market
0	29	Cummings-Ward	construction materials import/distribution, ir...	23
1	161	Maggio-Prohaska	construction materials import/distribution, ne...	20
2	175	Legros Group	construction materials import/distribution, st...	23
3	234	Koepp, Hessel and Runolfsson	construction materials import/distribution	15
4	241	Koch, Lang and Mertz	construction materials import/distribution, po...	12
5	308	Reichel and Sons	construction materials import/distribution	16
6	325	Marks Inc	construction materials import/distribution	19
7	364	Okuneva, Simonis and Hintz	construction materials import/distribution	24
8	378	Schiller-Bartoletti	construction materials import/distribution	17
9	379	Gerlach Group	construction materials import/distribution, el...	2
10	504	Herman, Turcotte and Osinski	construction materials import/distribution, ce...	16
11	565	Abernathy-Hayes	construction materials import/distribution, ir...	15
12	583	Kulas-Terry	construction materials import/distribution, co...	19
13	619	Kerluke, Barrows and Murazik	construction materials import/distribution, as...	19
14	707	Haley-Larson	construction materials import/distribution	14
15	715	Keebler, Deckow and Watsica	construction materials import/distribution	8
16	723	Stehr-Mitchell	construction materials import/distribution	9
17	808	Reilly, Hickle and Thiel	construction materials import/distribution, ne...	18
18	815	Schamberger and Sons	construction materials import/distribution, co...	8
		Grady, Kessler and	construction materials	

```
#Elimino columnas innecesarias para el relevamiento de informacion
df_Distribuidores.drop(['distributor activities','years in the construction market'],axis=1, inplace = True)
df_Distribuidores
```

	id	distributor
0	29	Cummings-Ward
1	161	Maggio-Prohaska
2	175	Legros Group
3	234	Koepp, Hessel and Runolfsson
4	241	Koch, Lang and Mertz
5	308	Reichel and Sons
6	325	Marks Inc
7	364	Okuneva, Simonis and Hintz
8	378	Schiller-Bartoletti
9	379	Gerlach Group
10	504	Herman, Turcotte and Osinski
11	565	Abernathy-Hayes
12	583	Kulas-Terry
13	619	Kerluke, Barrows and Murazik
14	707	Haley-Larson
15	715	Keebler, Deckow and Watsica
16	723	Stehr-Mitchell
17	808	Reilly, Hickle and Thiel
18	815	Schamberger and Sons
19	818	Grady, Kessler and Stokes
20	846	Kertzmann-Nolan
21	860	Kling Group
22	920	Denesik, Schmidt and Wyman
23	1016	Rosenbaum LLC
24	1055	Cummings, Kemmer and Walker
25	1062	Feil, MacGyver and Schamberger
26	1093	Davis, Kiehn and Heller
27	1104	Langworth Group
28	1169	Williamson, Bailey and McLaughlin
29	1183	Brekke-Stiedemann
30	1235	Tromp-Abshire

```
#Visualizo los nombres de las columnas
df_Distribuidores.columns
```

```
Index(['id', 'distributor'], dtype='object')
```

```
df_Distribuidores.rename(columns={'id':'Distribuidor','distributor':'Nombre Distribuidor'}, inplace=True)

```

```
df_Distribuidores
```

	Distribuidor	Nombre Distribuidor
0	29	Cummings-Ward
1	161	Maggio-Prohaska
2	175	Legros Group
3	234	Koepp, Hessel and Runolfsson
4	241	Koch, Lang and Mertz
5	308	Reichel and Sons
6	325	Marks Inc
7	364	Okuneva, Simonis and Hintz
8	378	Schiller-Bartoletti
9	379	Gerlach Group
10	504	Herman, Turcotte and Osinski
11	565	Abernathy-Hayes
12	583	Kulas-Terry
13	619	Kerluke, Barrows and Murazik
14	707	Haley-Larson
15	715	Keebler, Deckow and Watsica
16	723	Stehr-Mitchell
17	808	Reilly, Hickle and Thiel
18	815	Schamberger and Sons
19	818	Grady, Kessler and Stokes
20	846	Kertzmann-Nolan
21	860	Kling Group
22	920	Denesik, Schmidt and Wyman
23	1016	Rosenbaum LLC
24	1055	Cummings, Kemmer and Walker
25	1062	Feil, MacGyver and Schamberger
26	1093	Davis, Kiehn and Heller
27	1104	Langworth Group
28	1169	Williamson, Bailey and McLaughlin
29	1183	Brekke-Stiedemann
30	1235	Tromp-Abshire

```
df_Distribuidores.dtypes
```

```
Distribuidor            int64
Nombre Distribuidor    object
dtype: object
```

```
##################################################################################################################
#################################################################### LOCALIZACION ################################################################
##################################################################################################################
```

```
# CSV Localizacion
df_Localizacion= pd.read_csv(URL_localizacion)
# EXPLORO LOS TIPOS DE DATOS
print (df_Localizacion.dtypes)
```

```
PYid            int64
id            float64
location       object
department     object
activities     object
dtype: object
```

```
df_Localizacion
```

	PYid	id	location	department	activities
0	12	NaN	Bahía Negra	Alto Paraguay	food and beverage products manufacture, other ...
1	13	NaN	Capitán Carmelo Peralta	Alto Paraguay	food and beverage products manufacture, other ...
2	14	NaN	Fuerte Olimpo	Alto Paraguay	food and beverage products manufacture, other ...
3	15	NaN	Puerto Casado	Alto Paraguay	food and beverage products manufacture, other ...
4	16	NaN	Ciudad del Este	Alto Paraná	agriculture, livestock, hunting and related, f...
...	...	...	...	...	...
267	269	NaN	Veinticinco de Diciembre	San Pedro	food and beverage products manufacture, other ...
268	270	NaN	Villa del Rosario	San Pedro	food and beverage products manufacture, other ...

```
df_Localizacion.sort_values('PYid')
```

	PYid	id	location	department	activities
147	1	286.00	Itapé	Guairá	agriculture, livestock, hunting and related, t...
247	2	387.00	Villa Hayes	Presidente Hayes	commerce, steel,agriculture, livestock, huntin...
183	3	112.00	San Cosme y Damián	Itapúa	agriculture, livestock, hunting and related, t...
136	4	102.00	Tobatí	Cordillera	bricks, tiles and ceramics manufacture, craft ...
254	5	519.00	Guayaybi	San Pedro	agriculture, livestock, hunting and related
...	...	...	...	...	...
266	268	NaN	Unión	San Pedro	food and beverage products manufacture, other ...
267	269	NaN	Veinticinco de Diciembre	San Pedro	food and beverage products manufacture, other ...

```
#Verifico que no hayan datos duplicados y que me los sume (Me indica que la fila 19 se encuentra duplicada y la debo eliminar)
df_Localizacion['id'].duplicated()
```

```
0      False
1       True
2       True
3       True
4       True
       ...
267     True
268     True
269     True
270     True
271    False
Name: id, Length: 272, dtype: bool
```

```
#Observamos la Columna 1, FILA 44 Y 45 son columnas que no tienen un dato NaN que esta duplicado
#Muestro los datos duplicados por ID
df_Localizacion[df_Localizacion['id'].duplicated(keep=False)]
```

	PYid	id	location	department	activities
0	12	NaN	Bahía Negra	Alto Paraguay	food and beverage products manufacture, other ...
1	13	NaN	Capitán Carmelo Peralta	Alto Paraguay	food and beverage products manufacture, other ...
2	14	NaN	Fuerte Olimpo	Alto Paraguay	food and beverage products manufacture, other ...
3	15	NaN	Puerto Casado	Alto Paraguay	food and beverage products manufacture, other ...
4	16	NaN	Ciudad del Este	Alto Paraná	agriculture, livestock, hunting and related, f...
...	...	...	...	...	...
266	268	NaN	Unión	San Pedro	food and beverage products manufacture, other ...
267	269	NaN	Veinticinco de Diciembre	San Pedro	food and beverage products manufacture, other ...

```
#Elimino Todas las filas que no contienen datos
#axis=0 (filas) y axis=1(columnas)
df_Localizacion.dropna(axis=0, inplace = True)
```

```
df_Localizacion
```

	PYid	id	location	department	activities
118	7	409.00	Altos	Cordillera	agriculture, livestock, hunting and related, l...
136	4	102.00	Tobatí	Cordillera	bricks, tiles and ceramics manufacture, craft ...
142	11	523.00	Coronel Martínez	Guairá	agriculture, livestock, hunting and related
147	1	286.00	Itapé	Guairá	agriculture, livestock, hunting and related, t...
183	3	112.00	San Cosme y Damián	Itapúa	agriculture, livestock, hunting and related, t...
185	9	402.00	San Pedro del Paraná	Itapúa	agriculture, livestock, hunting and related
195	10	169.00	San Miguel	Misiones	craft production

```
df_Localizacion['id']=df_Localizacion['id'].astype(int)
```

```
df_Localizacion
```

	PYid	id	location	department	activities
118	7	409	Altos	Cordillera	agriculture, livestock, hunting and related, l...
136	4	102	Tobatí	Cordillera	bricks, tiles and ceramics manufacture, craft ...
142	11	523	Coronel Martínez	Guairá	agriculture, livestock, hunting and related
147	1	286	Itapé	Guairá	agriculture, livestock, hunting and related, t...
183	3	112	San Cosme y Damián	Itapúa	agriculture, livestock, hunting and related, t...
185	9	402	San Pedro del Paraná	Itapúa	agriculture, livestock, hunting and related
195	10	169	San Miguel	Misiones	craft production

```
#Distribuidores activos
df_Localizacion.rename(columns={'id':'Distribuidor'})
```

	PYid	Distribuidor	location	department	activities
118	7	409	Altos	Cordillera	agriculture, livestock, hunting and related, l...
136	4	102	Tobatí	Cordillera	bricks, tiles and ceramics manufacture, craft ...
142	11	523	Coronel Martínez	Guairá	agriculture, livestock, hunting and related
147	1	286	Itapé	Guairá	agriculture, livestock, hunting and related, t...
183	3	112	San Cosme y Damián	Itapúa	agriculture, livestock, hunting and related, t...
185	9	402	San Pedro del Paraná	Itapúa	agriculture, livestock, hunting and related
195	10	169	San Miguel	Misiones	craft production

```
#Exploracion de datos
# Combino los df_ventas y df_distribuidores mediante la columna Distribuidor (inner join)
datos_ventas_distribuidores=pd.merge(df_Ventas, df_Distribuidores , how='inner',on='Distribuidor')
datos_ventas_distribuidores.rename(columns={'Rubber':'Caucho'
,'Brass':'Laton'
,'Vinyl':'Vinilo'
,'Granite':'Granito'
,'Stone':'Piedra'
,'Brick':'Ladrillo'
,'Aluminum':'Aluminio'
,'Glass':'Vidrio'
,'Plexiglass':'Plexiglass'
,'Steel':'Acero'
,'Wood':'Madera'
,'Plastic':'Plastico'
}, inplace=True)
datos_ventas_distribuidores.sort_values('Distribuidor', ascending=True, inplace= True)
```

```
datos_ventas_distribuidores
```

	Distribuidor	Caucho	Laton	Vinilo	Granito	Piedra
42	29	65978582.24	36603264.50	57009912.25	16931446.04	41572788.80
23	161	41572788.80	35563624.11	27179878.86	79358855.35	1346769.34
12	175	26576776.52	22300838.58	1473437.08	41572788.80	65978582.24
30	234	11812951.32	11758005.07	18421797.13	43512922.94	16086016.33
13	241	59862809.01	55020982.80	78927599.01	55020982.80	54510203.61
34	308	18050385.12	41899590.44	33102840.61	20344007.84	45268636.86
40	325	36999867.09	6662942.92	76443998.86	45268636.86	18050385.12
27	364	53172624.14	33102840.61	177874267.01	81812951.32	42411396.23
24	378	16086016.33	11812951.32	18050385.12	51579748.25	21780180.58
3	379	79358855.35	90185311.22	45268636.86	54510203.61	59358855.35
17	504	20344007.84	1346769.34	22300838.58	35563624.11	27045333.20
8	565	57009912.25	53172624.14	36999867.09	59199680.31	78927599.01
0	583	54510203.61	45268636.86	51579748.25	21780180.58	26576776.52
37	619	43512922.94	20344007.84	55020982.80	42411396.23	11812951.32
10	707	59199680.31	59199680.31	9945371.16	55872547.77	43512922.94
38	715	9945371.16	41572788.80	6662942.92	1473437.08	55020982.80
18	723	77874267.01	51579748.25	32067534.68	11758005.07	57009912.25
39	808	3221765.13	10536882.23	35563624.11	46039695.30	22300838.58
22	815	51579748.25	46239695.30	53172624.14	27179878.86	25837100.49

```
columnas=list(datos_ventas_distribuidores.columns)
columnas
```

```
['Distribuidor',
 'Caucho',
 'Laton',
 'Vinilo',
 'Granito',
```

```
 'Piedra',
 'Ladrillo',
 'Aluminio',
 'Vidrio',
 'Plexiglass',
 'Acero',
 'Madera',
 'Plastico',
 'Nombre Distribuidor']
```

```
#cambio el orden de nombre de distribuidor asi me queda al lado de distribuidor
columnas[1],columnas[13]=columnas[13],columnas[1]
columnas
```

```
['Distribuidor',
 'Nombre Distribuidor',
 'Laton',
 'Vinilo',
 'Granito',
 'Piedra',
 'Ladrillo',
 'Aluminio',
 'Vidrio',
 'Plexiglass',
 'Acero',
 'Madera',
 'Plastico',
 'Caucho']
```

```
#Exploracion de datos
# Combino los df_Exportacion y df_distribuidores mediante la columna Distribuidor (inner join)
datos_exportacion_distribuidores=pd.merge(df_Exportacion, df_Distribuidores , how='inner',on='Distribuidor')
datos_exportacion_distribuidores.rename(columns={'Rubber':'Caucho'
,'Brass':'Laton'
,'Vinyl':'Vinilo'
,'Granite':'Granito'
,'Stone':'Piedra'
,'Brick':'Ladrillo'
,'Aluminum':'Aluminio'
,'Glass':'Vidrio'
,'Plexiglass':'Plexiglass'
,'Steel':'Acero'
,'Wood':'Madera'
,'Plastic':'Plastico'
}, inplace=True)
datos_exportacion_distribuidores.sort_values('Distribuidor', ascending=True, inplace= True)
```

```
columnas=list(datos_exportacion_distribuidores.columns)
columnas
```

```
['Distribuidor',
 'Caucho',
 'Laton',
 'Vinilo',
 'Granito',
 'Piedra',
 'Ladrillo',
 'Aluminio',
 'Vidrio',
 'Plexiglass',
```